

✓ 1. 데이터 준비

```
import pandas as pd

# 가상의 데이터 생성 (실습 편의상)
data = {'X': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
        'y': [2.1, 3.9, 6.2, 7.8, 10.1, 12.0, 14.3, 16.0, 18.2, 20.0]}
df = pd.DataFrame(data)

print("데이터프레임 상위 5개 행:")
print(df.head())
print("\n데이터프레임 정보:")
df.info()
print("\n데이터프레임 통계 요약:")
print(df.describe())
```

데이터프레임 상위 5개 행:

	X	y
0	1	2.1
1	2	3.9
2	3	6.2
3	4	7.8
4	5	10.1

데이터프레임 정보:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 2 columns):
#   Column  Non-Null Count  Dtype
---  -
0    X      10 non-null       int64
1    y      10 non-null       float64
dtypes: float64(1), int64(1)
memory usage: 292.0 bytes
```

데이터프레임 통계 요약:

	X	y
count	10.00000	10.00000
mean	5.50000	11.06000
std	3.02765	6.08645
min	1.00000	2.10000
25%	3.25000	6.60000
50%	5.50000	11.05000
75%	7.75000	15.57500
max	10.00000	20.00000

코드 설명:

- `import pandas as pd`: 데이터 분석을 위한 라이브러리 pandas를 불러옵니다.
 - `df = pd.DataFrame(data)`: 데이터를 DataFrame 형태로 만듭니다.
 - `df.head()`: 데이터의 첫 몇 줄을 보여줍니다. 데이터가 어떻게 생겼는지 빠르게 확인합니다.
 - `df.info()`: 데이터의 각 열(컬럼)에 대한 정보(데이터 타입, 결측치 여부)를 보여줍니다.
 - `df.describe()`: 각 열의 통계적 요약(평균, 표준편차, 최소/최대값 등)을 보여줍니다.
-

✓ 2. 데이터 분할

- 모델이 학습하지 않은 새로운 데이터에 대해 얼마나 잘 작동하는지 객관적으로 평가하기 위함입니다.
- 훈련 데이터로만 평가하면 과대적합 여부를 알 수 없습니다.

```
from sklearn.model_selection import train_test_split
import numpy as np

# 특성(X)과 타겟(y) 분리
X = df[['X']] # 특성은 2차원 배열 형태여야 합니다.
y = df['y']

# 훈련 세트와 테스트 세트로 분할 (80% 훈련, 20% 테스트)
X_train, X_test, y_train, y_test = \
    train_test_split(X, y, test_size=0.2, random_state=42)

print(f"훈련 세트 X 크기: {X_train.shape}")
print(f"훈련 세트 y 크기: {y_train.shape}")
print(f"테스트 세트 X 크기: {X_test.shape}")
print(f"테스트 세트 y 크기: {y_test.shape}")
```

```
훈련 세트 X 크기: (8, 1)
훈련 세트 y 크기: (8,)
테스트 세트 X 크기: (2, 1)
테스트 세트 y 크기: (2,)
```

코드 설명:

- `from sklearn.model_selection import train_test_split`: 데이터를 분할하는 함수를 불러옵니다.
 - `X = df[['X']], y = df['y']`: 데이터프레임에서 특성(독립 변수)과 타겟(종속 변수)을 분리합니다. X는 보통 2차원 배열 형태여야 합니다.
 - `train_test_split(X, y, test_size=0.2, random_state=42)`:
 - X, y: 분할할 데이터.
 - `test_size=0.2`: 전체 데이터의 20%를 테스트 세트로 사용하겠다는 의미입니다. (나머지 80%는 훈련 세트)
 - `random_state=42`: 데이터를 무작위로 섞을 때 사용하는 '시드' 값입니다. 이 값을 고정하면 코드를 다시 실행해도 항상 동일하게 분할됩니다. (재현성 확보)
-

- test_size 값을 0.3으로 변경하여 데이터를 다시 나눠보고, 분할된 X_train, X_test의 크기가 어떻게 변하는지 확인해 보세요.

```
X_train, X_test, y_train, y_test = \
    train_test_split(X, y, test_size=0.3, random_state=42)

print(f"훈련 세트 X 크기: {X_train.shape}")
print(f"훈련 세트 y 크기: {y_train.shape}")
print(f"테스트 세트 X 크기: {X_test.shape}")
print(f"테스트 세트 y 크기: {y_test.shape}")
```

```
훈련 세트 X 크기: (7, 1)
훈련 세트 y 크기: (7,)
테스트 세트 X 크기: (3, 1)
테스트 세트 y 크기: (3,)
```

✓ 3. 선형 회귀 모델 생성 및 훈련

선형 회귀 모델 객체를 만들고, 훈련 데이터를 사용하여 최적의 W와 b를 찾도록 학습시킵니다.

```
from sklearn.linear_model import LinearRegression

# 선형 회귀 모델 객체 생성
model = LinearRegression()
print("선형 회귀 모델 객체 생성 완료:", model)

# 블랙박스 들여다보기:
# model = LinearRegression() 한 줄은
# 메모리에 '선형 회귀를 수행할 준비가 된 빈 모델'을 만듭니다.
# 아직 어떤 데이터도 학습하지 않았기 때문에, w와 b 값은 정해지지 않았습니다.

# 모델 훈련 (학습)
# 이 과정에서 모델은 훈련 데이터를 사용하여 최적의 w와 b를 찾습니다.
model.fit(X_train, y_train)

print("\n모델 훈련 완료!")

선형 회귀 모델 객체 생성 완료: LinearRegression()

모델 훈련 완료!
```

코드 설명:

- from sklearn.linear_model import LinearRegression:
 - 사이킷런에서 선형 회귀 모델 클래스를 불러옵니다.
- model = LinearRegression():
 - LinearRegression 클래스의 인스턴스(객체)를 생성합니다. 이것이 바로 우리가 사용할 선형 회귀 모델입니다.
- model.fit(X_train, y_train):

- fit() 메서드는 모델을 **훈련(학습)**시키는 역할을 합니다.
- X_train (훈련 특성)과 y_train (훈련 타겟) 데이터를 사용하여, 모델은 이전에 설명했던 경사 하강법과 같은 내부 알고리즘을 통해 MSE를 최소화하는 최적의 W와 b 값을 찾아냅니다.

미니 실습 2:

- 모델 훈련 후, model.coef_와 model.intercept_를 출력하여 최적화된 W (가중치/기울기)와 b (편향/절편) 값을 확인해 보세요.

```
print(f"최적화된 가중치 (W): {model.coef_[0]:.4f}")
print(f"최적화된 편향 (b): {model.intercept_: .4f}")
```

- 이 값들이 우리가 처음에 만든 가상 데이터의 실제 W=3, b=4와 얼마나 비슷한지 비교해 보세요. (노이즈 때문에 정확히 같지는 않습니다.)

```
print(f"최적화된 가중치 (W): {model.coef_[0]:.4f}")
print(f"최적화된 편향 (b): {model.intercept_: .4f}")
```

최적화된 가중치 (W): 1.9957
최적화된 편향 (b): 0.0862

✓ 4. 예측 수행

훈련된 모델을 사용하여 새로운 데이터(X_test)에 대한 예측 값을 만듭니다.

```
# 훈련된 모델로 테스트 데이터에 대한 예측 수행
y_pred = model.predict(X_test)

print(X_test[:5])
print("\n테스트 데이터에 대한 예측값 (상위 5개):")
print(y_pred[:5])
print("\n테스트 데이터의 실제값 (상위 5개):")
print(y_test.head())
```

```
      X
8      9
1      2
5      6
```

테스트 데이터에 대한 예측값 (상위 5개):
[18.04741379 4.07758621 12.06034483]

테스트 데이터의 실제값 (상위 5개):
8 18.2
1 3.9
5 12.0
Name: y, dtype: float64

코드 설명:

- y_pred = model.predict(X_test):

- predict() 메서드는 훈련된 모델이 새로운 입력 데이터(X_test)에 대해 예측 값을 생성하도록 합니다.

✓ 5. 모델 평가

- 모델이 얼마나 잘 예측하는지 객관적인 지표로 확인하고, 다른 모델과 비교할 수 있도록 합니다.
- 모델이 실제 문제를 얼마나 잘 해결하는지 파악하기 위함입니다.
- 모델을 개선하거나 다른 모델을 선택할 때 기준이 됩니다.

```
from sklearn.metrics import mean_squared_error, r2_score

# 테스트 세트에 대한 MSE 계산
mse = mean_squared_error(y_test, y_pred)
print(f"\n테스트 세트 평균 제곱 오차 (MSE): {mse:.4f}")

# 테스트 세트에 대한 R2 Score 계산
r2 = r2_score(y_test, y_pred)
print(f"테스트 세트 R2 Score: {r2:.4f}")
```

```
테스트 세트 평균 제곱 오차 (MSE): 0.0195
테스트 세트 R2 Score: 0.9994
```

코드 설명:

- from sklearn.metrics import ...:
 - 모델 평가 지표를 제공하는 사이킷런 모듈을 불러옵니다.
- mean_squared_error(y_test, y_pred):
 - 실제 값(y_test)과 예측 값(y_pred) 사이의 평균 제곱 오차를 계산합니다.
 - 작을수록 좋습니다. 오차가 작다는 것은 예측이 실제 값과 가깝다는 의미입니다.
- r2_score(y_test, y_pred):
 - R2 Score (결정 계수)를 계산합니다.
 - 모델이 데이터를 얼마나 잘 설명하는지 나타내는 지표입니다.
 - 0이면 모델이 데이터를 전혀 설명하지 못하고, 1이면 완벽하게 설명합니다.

미니 실습 3:

- 훈련 세트(y_train, model.predict(X_train))에 대해서도 MSE와 R2 Score를 계산해보고, 테스트 세트 결과와 비교해 보세요.
- 훈련 세트에서는 잘 맞는데 테스트 세트에서는 왜 성능이 떨어질까요? (과대적합 가능성)
- 두 값이 비슷하다면 어떤 의미일까요? (모델이 잘 일반화되었다는 의미)

```
# 훈련 세트에 대한 MSE 계산
mse = mean_squared_error(y_train, model.predict(X_train))
print(f"\n테스트 세트 평균 제곱 오차 (MSE): {mse:.4f}")
```

```
# 훈련 세트에 대한 R2 Score 계산
r2 = r2_score(y_train, model.predict(X_train))
print(f"테스트 세트 R2 Score: {r2:.4f}")
```

테스트 세트 평균 제곱 오차 (MSE): 0.0220
테스트 세트 R2 Score: 0.9993